

Integração do Github Project com OKRs: Personalizações e Justificativas

Rodrigo Souza dos Santos

2026-03-11

Índice

Preface	4
1 Uso de GitHub para gestão de OKRs e execução técnica	5
2 Uso de GitHub Projects para gestão de OKRs	6
2.1 Descrição	6
2.2 Como ele é usado no modelo OKR	6
2.3 Justificativa	7
2.3.1 Integração direta com desenvolvimento	7
2.3.2 Visibilidade estratégica	7
2.3.3 Transparência organizacional	8
3 Uso de Issue Types para estruturação de OKRs	9
3.1 Descrição	9
3.2 Estrutura organizacional dos OKRs	9
3.3 Objective (O)	10
3.4 Key Result (KR)	10
3.5 Epic	11
3.6 Story	11
3.7 Task	11
4 Por que labels não são suficientes	12
4.1 Limitações do uso exclusivo de labels	12
4.1.1 Falta de exclusividade semântica	12
4.1.2 Falta de semântica estrutural	13
4.1.3 Baixa confiabilidade para automação	13
4.1.4 Falta de padronização	13
5 Diferença conceitual entre Issue Types e Labels	14
5.1 Analogia organizacional	14
6 Como Issue Types serão usados no GitHub Projects	15
6.1 Visão estratégica	15
6.2 Visão de execução	15
6.3 Visão completa	16

7	Papel complementar das labels	17
8	Uso de GitHub Actions para automação do processo	18
8.1	Descrição	18
8.2	Aplicação no fluxo de OKRs	18
8.2.1	Validação estrutural	18
8.2.2	Atualização automática de progresso	18
8.2.3	Atualização automática de dashboards	19
8.3	Justificativa	19
8.3.1	Redução de trabalho manual	19
8.3.2	Confiabilidade dos dados	19
8.3.3	Integração com fluxo de engenharia	19
9	Conclusão	20
	References	22

Preface

This is a Quarto book.

To learn more about Quarto books visit <https://quarto.org/docs/books>.

1 Uso de GitHub para gestão de OKRs e execução técnica

Esta seção descreve a abordagem adotada para implementar **OKRs (Objectives and Key Results)** dentro do fluxo de desenvolvimento usando **GitHub**, combinando:

- **GitHub Projects**
- **Issue Types**
- **Labels**
- **GitHub Actions**

Essa integração permite conectar **planejamento estratégico, execução técnica e monitoramento de progresso** dentro de uma única plataforma.

2 Uso de GitHub Projects para gestão de OKRs

2.1 Descrição

GitHub Projects é um sistema integrado ao repositório que permite organizar issues em **quadros, tabelas e dashboards**, adicionando campos estruturados como status, responsável, prioridade e progresso.

No contexto de OKRs, ele funciona como um **painel central de acompanhamento estratégico**, conectando diretamente os objetivos organizacionais às atividades de engenharia.

2.2 Como ele é usado no modelo OKR

A estrutura implementada é baseada na seguinte hierarquia:

```
Objective (O)
  ↓
Key Result (KR)
  ↓
Epic
  ↓
Story
  ↓
Task
```

Essa hierarquia conecta **direção estratégica** com **trabalho técnico executável**.

No **GitHub Projects**, isso permite:

- visualizar progresso dos OKRs
- acompanhar execução técnica
- identificar dependências
- monitorar evolução das iniciativas

Exemplo de organização:

Tipo	Item	Status
Objective	Automatizar inspeções submarinas	Em andamento
Key Result	Aumentar precisão do modelo para 90%	Em andamento
Epic	Pipeline de visão computacional	Planejado
Story	Treinar modelo de detecção	Em andamento

2.3 Justificativa

2.3.1 Integração direta com desenvolvimento

Ao contrário de ferramentas externas de gestão, o Projects está **diretamente integrado ao repositório**, permitindo:

- ligação com código
- ligação com pull requests
- rastreabilidade completa

Isso reduz desalinhamentos entre **planejamento e execução**.

2.3.2 Visibilidade estratégica

OKRs são instrumentos de **direcionamento organizacional**.

GitHub Projects permite visualizar:

- objetivos ativos
- resultados-chave associados
- progresso de iniciativas

O repositório passa a funcionar como **painel de execução estratégica**.

2.3.3 Transparência organizacional

Como o projeto é visível para toda a equipe:

- melhora comunicação
 - reduz dependência de relatórios manuais
 - aumenta alinhamento entre áreas técnicas e estratégicas.
-

3 Uso de Issue Types para estruturação de OKRs

3.1 Descrição

Issue Types permitem classificar issues de forma estruturada, definindo o papel que cada issue desempenha no processo de planejamento e execução.

No modelo adotado, os tipos utilizados são:

Tipo	Função
Objective	objetivo estratégico
Key Result	métrica mensurável
Epic	grande iniciativa
Story	funcionalidade
Task	implementação técnica

Esses tipos refletem os diferentes níveis da hierarquia de planejamento.

3.2 Estrutura organizacional dos OKRs

A estrutura implementada segue o modelo:

```
Objective
  ↓
Key Result
  ↓
Epic
  ↓
Story
  ↓
Task
```

Isso permite navegar da estratégia até o trabalho de engenharia.

3.3 Objective (O)

Representa o **resultado estratégico desejado**.

Características:

- qualitativo
- orientado a impacto
- define direção organizacional

Exemplo:

Melhorar eficiência das inspeções submarinas com AUV

3.4 Key Result (KR)

Representa **uma métrica que mede progresso do Objective**.

Características:

- quantitativo
- mensurável
- verificável

Exemplo:

Aumentar precisão do modelo de detecção de corrosão de 70% para 90%

3.5 Epic

Representa uma **iniciativa técnica significativa** necessária para alcançar um KR.

Exemplo:

Desenvolver pipeline de processamento de vídeo submarino

3.6 Story

Representa uma **funcionalidade específica ou requisito técnico**.

Exemplo:

Detectar corrosão em imagens de inspeção submarina

3.7 Task

Representa **uma atividade técnica concreta**.

Exemplo:

Implementar script de extração de frames de vídeo

4 Por que labels não são suficientes

Embora labels sejam úteis para classificação, **elas não substituem Issue Types** em uma estrutura de OKRs.

4.1 Limitações do uso exclusivo de labels

4.1.1 Falta de exclusividade semântica

Uma issue pode ter múltiplas labels.

Exemplo:

```
okr  
ml  
priority-high  
objective  
key-result
```

Nada impede que uma mesma issue tenha **labels conflitantes**.

Issue Types evitam isso porque **cada issue possui apenas um tipo principal**.

4.1.2 Falta de semântica estrutural

Labels são apenas etiquetas.

Elas não indicam:

- papel da issue
- posição na hierarquia
- relação com planejamento estratégico

Issue Types definem **categorias formais de trabalho**.

4.1.3 Baixa confiabilidade para automação

Labels podem:

- ser removidas
- ser aplicadas incorretamente
- existir duplicadas

Isso reduz confiabilidade para automação.

Issue Types fornecem **estrutura confiável para workflows automatizados**.

4.1.4 Falta de padronização

Labels podem ser criadas livremente, gerando inconsistências como:

objective
objectives
okr-objective
goal

Issue Types garantem **padronização organizacional**.

5 Diferença conceitual entre Issue Types e Labels

Aspecto	Issue Types	Labels
Função	definir papel estrutural	categorizar conteúdo
Quantidade por issue	1 principal	múltiplas
Estrutura organizacional	sim	não
Semântica	forte	fraca
Automação	confiável	limitada

5.1 Analogia organizacional

Issue Types são comparáveis a **tipos de documento**:

Contrato
Relatório
Proposta

Labels são comparáveis a **tags de classificação**:

financeiro
urgente
cliente-A

Ambos são úteis, mas cumprem papéis diferentes.

6 Como Issue Types serão usados no GitHub Projects

No **GitHub Projects**, Issue Types permitem criar diferentes visões do trabalho.

6.1 Visão estratégica

Filtro:

```
type:objective  
type:key-result
```

Mostra apenas OKRs.

6.2 Visão de execução

Filtro:

```
type:epic  
type:story  
type:task
```

Mostra backlog técnico.

6.3 Visão completa

Objective

→ Key Result

→ Epic

→ Story

→ Task

Isso permite navegar do planejamento até a implementação.

7 Papel complementar das labels

Labels continuam sendo úteis para **classificação temática ou operacional**.

Exemplos:

```
ml
computer-vision
dataset
auv
subsea
research
priority-high
```

Assim:

Elemento	Função
Issue Type	papel estrutural da issue
Labels	categorização temática

8 Uso de GitHub Actions para automação do processo

8.1 Descrição

GitHub Actions permite automatizar processos baseados em eventos do repositório.

Workflows podem ser executados quando:

- issues são criadas
 - issues são editadas
 - issues são fechadas
 - pull requests são abertos
-

8.2 Aplicação no fluxo de OKRs

8.2.1 Validação estrutural

Quando um KR é criado, um workflow pode verificar se ele referencia um Objective válido.

8.2.2 Atualização automática de progresso

O progresso de um KR pode ser calculado com base em:

- tarefas concluídas
 - checklists
 - issues relacionadas
-

8.2.3 Atualização automática de dashboards

Actions podem atualizar:

- campos no GitHub Projects
 - relatórios de progresso
 - comentários automáticos
-

8.3 Justificativa

8.3.1 Redução de trabalho manual

Sem automação seria necessário:

- atualizar progresso manualmente
- gerar relatórios
- verificar consistência

Actions eliminam essas tarefas repetitivas.

8.3.2 Confiabilidade dos dados

A automação garante que métricas derivem diretamente do trabalho realizado.

8.3.3 Integração com fluxo de engenharia

Como workflows são acionados por eventos do repositório, o progresso dos OKRs permanece **diretamente conectado ao desenvolvimento real.**

9 Conclusão

A combinação de:

- **GitHub Projects**
- **Issue Types**
- **Labels**
- **GitHub Actions**

permite criar um sistema em que:

- OKRs definem direção estratégica
- issues estruturam o trabalho
- actions automatizam acompanhamento
- projects fornecem visibilidade

Essa abordagem conecta **planejamento estratégico, execução técnica e monitoramento de progresso**, garantindo **rastreabilidade, transparência e escalabilidade organizacional**.

10

References